

IDV-ASAS WebUI 版

Identity V Area Selection Assistance System — Web UI Version 《第五人格》区域选择辅助系统 — Web 界面版

IDV-ASAS WebUI 版是基于 Rust + Axum 的本地 Web 应用，通过调用 OpenAI 兼容的大语言模型，为《第五人格》排位赛中的求生者阵营提供**天赋选择与区域选择联合推荐方案**。与桌面 GUI 版（`app/`）相比，后端逻辑完全相同，仅将 eframe/egui 桌面界面替换为浏览器 Web 界面。

目录

- [与桌面版的区别](#)
- [功能概述](#)
- [环境要求](#)
- [快速开始](#)
- [配置指南](#)
- [使用流程](#)
- [演示用例](#)
- [历史管理](#)
- [常见问题](#)
- [项目结构](#)

与桌面版的区别

对比项	桌面版 (<code>app/</code>)	WebUI 版 (<code>webui-ver/</code>)
界面技术	<code>eframe / egui</code> (Rust 原生 GUI)	HTML / CSS / JavaScript (浏览器)
后端模块	<code>api.rs</code> , <code>config.rs</code> , <code>checker.rs</code> , <code>positions.rs</code> , <code>prompt.rs</code> , <code>history.rs</code> , <code>parser.rs</code>	完全相同 (原样复制, 未做任何修改)
渲染方式	<code>wgpu</code> (Vulkan/GL)	浏览器原生渲染
图片合成	Rust <code>image</code> crate 像素级 alpha 混合	HTML5 Canvas <code>drawImage</code>
界面尺寸	固定 1280×720	自适应浏览器窗口
中文支持	内嵌 <code>Deng.ttf</code>	浏览器系统字体 / Google Fonts
窗口依赖	X11 (WSL2 需 WSLg)	无 (仅需浏览器)
运行方式	<code>cargo run</code> 后弹出桌面窗口	<code>cargo run</code> 后浏览器打开 <code>http://127.0.0.1:3000</code>
架构	单体 (UI + 逻辑同进程)	本地 Web 服务 (Axum 服务端 + 浏览器前端)

总结: WebUI 版移除了对图形窗口系统的依赖, 在无 X11/Wayland 的服务器环境 (如 SSH、Docker) 中也能使用, 同时保留全部后端功能不变。

功能概述

- **天赋 + 选点联合推荐:** 不止推荐选点, 还推荐每个角色的天赋搭配, 因天赋影响选点策略 (牵制天赋可占危险点位, 运营天赋需放安全位)
- **队伍联动分析:** 接入角色联动关系数据库, 联动方放在邻近点位; 为每位玩家展示全队方案
- **专属数据库:** 51 名角色六维属性 + 9 张地图选点数据, 注入模型上下文, 弥补通用模型对游戏机制理解不足
- **可视化展示:** 地图选点底图上叠加角色头像 (HTML5 Canvas), 右侧展示天赋图标, 下方配以文字分析
- **可打断调用:** 模型分析过程中支持随时打断, 避免网络异常时无限等待
- **历史续局:** 保存会话历史, 导入后自动继承全局需求与累计统计
- **多服务商兼容:** 支持任意 OpenAI 兼容接口 (OpenAI / DeepSeek / SiliconFlow / 自建网关等)
- **无需图形环境:** 仅需浏览器, 适合 SSH、Docker、无桌面环境的服务器

环境要求

项目	要求
操作系统	Linux / macOS / Windows (任何能运行 Rust 的系统)
Rust 工具链	cargo 1.75+ (建议 1.97+)
浏览器	任意现代浏览器 (Chrome、Firefox、Edge 等)
网络	需访问模型 API 服务; 浏览器需访问 127.0.0.1:3000
模型接口	任意 OpenAI 兼容的 chat/completions 服务

与桌面版区别: 无需 X11/Wayland 图形库, 无需 libx11-dev 等系统依赖, 无需 WSLg。

快速开始

1. 确认资源目录完整

项目根目录下需包含以下结构:

```
IDV-ASAS/
├─ webui-ver/                                # WebUI 版程序目录
│   ├─ Cargo.toml
│   ├─ static/                               # 前端静态文件
│   │   ├─ index.html
│   │   ├─ style.css
│   │   └─ app.js
│   ├─ config.json                          # 运行时生成: API 配置 (明文)
│   ├─ history/                             # 运行时生成: 会话历史 JSON
│   └─ src/                                 # Rust 源代码
├─ resources/                               # 后端数据资料
│   ├─ AGENTS.md
│   ├─ README.md
│   ├─ characters/                          # 51 个角色数据 (.md)
│   └─ maps/                               # 9 张地图数据 (.md)
├─ icons/                                   # 前端图标资源
│   ├─ maps/                               # 地图概念图 (PNG)
│   ├─ area-selection/                     # 区域选点底图 (PNG)
│   ├─ characters/                         # 角色头像 (PNG, 120x120)
│   ├─ personas/                           # 天赋图标 (PNG)
│   └─ positions/                           # 选点坐标数据 (.md)
└─ DICTIONARY.md                           # 中英文字典
```

2. 编译

```
cd webui-ver
cargo build --release
```

首次编译需要联网拉取依赖 (axum、tokio、tower-http、ureq 等), 耗时约 1~2 分钟。后续编译可离线进行。

3. 运行

```
cargo run --release
```

或直接运行编译产物：

```
./target/release/idv-asas-webui
```

程序启动后终端输出：

```
[IDV-ASAS webUI] 服务已启动: http://127.0.0.1:3000
[IDV-ASAS webUI] 在浏览器中打开上述地址即可使用
```

在浏览器中打开 `http://127.0.0.1:3000` 即可进入界面。

提示：服务仅监听 `127.0.0.1`（本机回环地址），不对外网暴露。如需远程访问，可通过 SSH 端口转发：`ssh -L 3000:127.0.0.1:3000 user@server`。

配置指南

程序启动后浏览器打开的第一步是配置 API 接口。有两种方式：**页面内配置**或**手动编辑 config.json**。

方式一：页面内配置（推荐）

首次启动（无 config.json）

页面直接显示配置表单，依次填写：

字段	说明	示例
API Base URL	服务商端点，填到 <code>/v1</code> 即可	<code>https://api.deepseek.com/v1</code>
API Key	密钥（输入框隐藏显示）	<code>sk-xxxxxxx</code>
模型名称	由服务商决定	<code>deepseek-chat</code>
温度 temperature	0~2 之间，默认 0.7	<code>0.7</code>
max_tokens	输出长度限制，留空 = 不限制	<code>4096</code>
输入 token 单价	人民币元 / 每百万 tokens	<code>2</code>
输出 token 单价	人民币元 / 每百万 tokens	<code>8</code>

勾选"保存为默认配置"（默认已勾选），点击 **确认** 即可。

非首次启动（有 config.json）

页面显示当前配置摘要（端点、模型、温度、单价），询问"是否沿用默认配置？"

- 点击 **是**：直接沿用，进入下一步
- 点击 **否**：展开编辑表单，修改后保存

方式二：手动编辑 config.json

配置文件位于 `webui-ver/config.json`，格式如下：

```
{
  "base_url": "https://api.deepseek.com/v1",
  "api_key": "sk-xxxxxxx",
  "model": "deepseek-chat",
  "temperature": 0.7,
  "max_tokens": null,
  "price_input_per_m": 2.0,
  "price_output_per_m": 8.0
}
```

字段	类型	说明
<code>base_url</code>	string	服务商端点，填到 <code>/v1</code>
<code>api_key</code>	string	Bearer 认证密钥
<code>model</code>	string	模型名称
<code>temperature</code>	float	0~2，默认 0.7
<code>max_tokens</code>	int / null	输出上限， <code>null</code> = 不限制
<code>price_input_per_m</code>	float	输入单价（元/百万tokens）
<code>price_output_per_m</code>	float	输出单价（元/百万tokens）

⚠ **安全提示：** `config.json` 中 API Key 为**明文存储**。建议将 `webui-ver/config.json` 和 `webui-ver/history/` 加入 `.gitignore`，切勿提交到代码仓库。

兼容的服务商

任意 OpenAI 兼容的 `chat/completions` 接口均可使用：

服务商	Base URL	模型示例
OpenAI	<code>https://api.openai.com/v1</code>	<code>gpt-4o-mini</code>
DeepSeek	<code>https://api.deepseek.com/v1</code>	<code>deepseek-chat</code>
SiliconFlow	<code>https://api.siliconflow.cn/v1</code>	<code>deepseek-ai/DeepSeek-V3</code>
自建网关	<code>http://localhost:8080/v1</code>	自定义

端点会自动补全：输入 `https://api.xxx.com/v1` 会自动拼接为 `.../v1/chat/completions`；若 URL 已包含 `/chat/completions` 则不重复追加。

使用流程

程序共 8 个步骤，完整流程如下：

第0步 配置API → 第1步 全局需求 → 第2步 选地图 → 第3步 选4角色
→ 第4步 当局需求 → 第5步 模型分析 → 第6步 展示方案
→ 第7步 对局结束 → 第8步 下一局? → (是：回到第2步 / 否：保存历史 → 退出)

第 0 步：配置 API 接口

见上方[配置指南](#)。

第 1 步：输入全局用户需求

在文本框中输入对每一局均生效的全局需求。

例如：

队伍需要携带两个化险为夷，优先保护羸弱角色。

若无全局需求，直接点击 **确定**。

第 2 步：选择当局地图

界面展示 3×3 地图概念图网格，共 9 张排位地图：

军工厂	圣心医院	红教堂
湖景村	月亮河公园	里奥的回忆
永眠镇	唐人街	不归林

点击地图图片选择，选中后以**黄色高亮框**标识。再次点击可取消，点击其他地图可切换。选定后点击 **确认**。

第 3 步：选择当局角色

按 `DICTIONARY.md` 顺序排列共 51 名角色，每页 18 格（左右各 3×3），共 3 页。点击角色头像选择，选中以黄色高亮框标识。选满 4 个后其余角色置灰。右侧已选角色栏实时更新。确认后进入下一步。

第 4 步：输入当局用户需求

输入仅对当前局生效的特殊需求。

例如：

本局医生想打野，不要放安全点。

若无当局需求，直接点击 **确定**，开始调用模型。

第 5 步：调用模型分析

显示加载动画和"正在调用模型进行分析....."。此过程中可随时点击 **打断** 停止。

- 正常完成 → 自动进入结果展示
- 打断/网络错误 → 进入错误界面，可选择重试或退出

第 6 步：展示推荐方案

左右布局展示：

- **左侧**：地图选点底图 + 4 名角色头像按推荐位置叠加（HTML5 Canvas 合成）
- **右侧上**：4 行角色头像 + 天赋图标 + 选点编号
- **右侧中**：文字分析（不超过 100 字）
- **右侧下**：◀▶ 翻页按钮 + 确认使用 按钮

翻页浏览多套方案，确认后界面锁定。

第 7 步：等待对局结束

完成游戏对局后点击 **对局结束**，显示本局 Token 用量与费用及会话累计统计。

第 8 步：是否开启下一对局

- **是**：返回第 2 步（地图选择），全局需求保留
- **否**：进入历史保存询问 → 退出

演示用例

以下是一次完整使用过程的示例。

场景

排位赛中，你方求生者阵容为：**佣兵、古董商、作曲家、空军**，地图为**军工厂**。

操作步骤

1. 启动程序

```
cd webui-ver && cargo run --release
```

浏览器打开 `http://127.0.0.1:3000`。

2. 配置 API（首次启动）

字段	填写内容
API Base URL	https://api.deepseek.com/v1
API Key	sk-your-key-here
模型名称	deepseek-chat
温度	0.7
max_tokens	(留空)
输入单价	2
输出单价	8

点击 **确认**。

3. **全局需求**（第 1 步）

输入： 队伍需要携带两个化险为夷，优先保护羸弱角色。 → 点击 **确定**

4. **选择地图**（第 2 步）

点击 **军工厂** 概念图 → 点击 **确认**

5. **选择角色**（第 3 步）

在角色网格中依次点击 **佣兵、古董商、作曲家、空军** → 点击 **确认**

6. **当局需求**（第 4 步）

输入： 本局作曲家想打野，不要放安全点。 → 点击 **确定**

7. **模型分析**（第 5 步）

等待模型返回结果（通常 10~30 秒）。

8. **查看方案**（第 6 步）

模型返回 2~3 套方案，例如：

方案 1：
5 号选点：佣兵：回光返照、化险为夷
2 号选点：古董商：回光返照、飞轮效应
6 号选点：作曲家：回光返照、膝跳反射
8 号选点：空军：回光返照、化险为夷
具体描述及分析：佣兵占中场抗压牵制，古董商守小树林稳定破译，作曲家与空军分守大门与沙包，整体均衡。

- 左侧地图上显示 4 个角色头像在对应选点位置
- 右侧显示每人天赋图标
- 用 ◀▶ 翻页浏览不同方案
- 选定方案 1 后点击 **确认使用**

9. **对局结束**（第 7 步）

游戏结束后点击 **对局结束**，查看：

本局 Token 用量

输入：12850 输出：420 总计：13270

本局费用：¥0.0604

会话累计

总输入：12850 总输出：420 总费用：¥0.0604

10. 下一局（第 8 步）

点击 **是** → 返回地图选择，全局需求保留 → 继续下一局

或点击 **否** → 保存历史 → 退出

历史管理

保存

退出程序前（无论是完成对局不继续、API 错误退出、还是关闭浏览器），页面都会询问是否保存会话历史：

- **保存** → 询问是否命名
 - 输入名称 → 以该名称保存为 `webui-ver/history/{名称}.json`
 - 留空 → 以当前时间命名（如 `20260905-003530.json`）
- **不保存** → 舍弃，显示"会话已结束"

关闭浏览器标签页时，浏览器会弹出"确认离开"提示（若有未保存会话）。

导入

启动程序完成 API 配置后，若 `webui-ver/history/` 下存在历史文件，会询问是否导入：

- **是** → 展示历史列表（模型、局数、累计 Token、费用、全局需求摘要）→ 点击列表项导入
- **否** → 跳过，新开会话

导入后：

- 全局需求与累计统计自动继承
- 直接进入第 2 步（地图选择），无需重新输入全局需求
- 历史局以紧凑形式进入模型上下文，避免 Token 膨胀
- 可更换 API 配置（如换模型续开）

手动维护

历史记录为 `webui-ver/history/` 目录下的普通 JSON 文件：

- **查看**：用文本编辑器打开
- **备份**：复制 JSON 文件
- **删除**：直接删除文件
- **重命名**：重命名文件即可，文件名即显示名称

常见问题

问题	处理方法
HTTP 401	API Key 错误或未填写
HTTP 404	Base URL 填错，应形如 <code>https://域名/v1</code>
HTTP 400 temperature参数非法	温度已自动四舍五入到 2 位小数；若仍报错，检查 config.json
Connection refused / 超时	网络不通或地址端口错误
等待模型响应超时	120 秒未收到任何输出，检查网络或模型服务状态
想更换模型	下次启动在第 0 步选择"否"重新配置，或手动编辑 config.json
想修改全局需求	新会话重新输入；或手动编辑 history/ 下对应 JSON 的 <code>global_req</code> 字段后加载
误点了打断	在错误界面点击"重试"即可重新生成
模型输出不合规	程序会自动重试 3 次；若持续不合规，提示"您的模型与该项目不适配"
刷新页面后状态丢失	服务端会话状态保留在内存中，但前端会回到配置步骤。可重新配置后直接进入历史导入恢复
页面无法访问	确认终端显示"服务已启动"；检查 3000 端口是否被占用
远程服务器使用	通过 SSH 端口转发： <code>ssh -L 3000:127.0.0.1:3000 user@server</code>

项目结构

```
IDV-ASAS/
├── README.md           # 顶层 README（桌面版）
├── AGENTS.md           # 顶层使用流程与实现指南
├── DICTIONARY.md       # 角色/地图/天赋中英文字典
├── app/
├── webui-ver/
│   ├── Cargo.toml      # Rust 依赖声明
│   ├── README_WEBUI.md # 本文件
│   ├── DESIGN_WEBUI.md # WebUI 版实现原理文档
│   ├── config.json     # 运行时生成：API 配置（明文）
│   ├── history/        # 运行时生成：会话历史 JSON
│   ├── static/         # 前端静态文件
│   │   ├── index.html  # 主页面（全部步骤界面）
│   │   ├── style.css   # 样式（深黑蓝 + 黄色主题）
│   │   └── app.js       # 前端逻辑（步骤流程 + Canvas 合成）
│   └── src/
│       ├── main.rs      # web 服务入口：路由、状态管理、SSE 流式分析
│       ├── api.rs       # ← 与桌面版完全相同
│       └── config.rs    # ← 与桌面版完全相同
```

```
|      |— checker.rs          # ← 与桌面版完全相同
|      |— positions.rs       # ← 与桌面版完全相同
|      |— prompt.rs         # ← 与桌面版完全相同
|      |— history.rs        # ← 与桌面版完全相同
|      |— parser.rs         # ← 与桌面版完全相同
|— resources/               # 后端数据资料
|  |— AGENTS.md             # 后端流程说明
|  |— README.md            # 求生者属性与天赋总纲
|  |— characters/          # 51 个角色数据 (.md)
|  |— maps/                # 9 张地图数据 (.md)
|— icons/                  # 前端图标资源
  |— AGENTS.md             # 前端流程说明
  |— maps/                 # 地图概念图 (9 PNG)
  |— area-selection/       # 区域选点底图 (9 PNG)
  |— characters/           # 角色头像 (51 PNG, 120×120)
  |— personas/            # 天赋图标 (4 PNG)
  |— positions/            # 选点坐标数据 (9 .md)
```

注意事项

1. 服务监听 `127.0.0.1:3000`，仅限本机访问，不对外网暴露。
2. 所有展示文字均为中文。
3. 鼠标用于选择操作，键盘用于文本输入（需求、命名等）。
4. `webui-ver/config.json` 和 `webui-ver/history/` 为运行时产物，建议加入 `.gitignore`。
5. API Key 明文存储于 `config.json`，请注意保管，切勿分享或提交到代码仓库。
6. 后端 7 个 Rust 模块与桌面版 (`app/src/`) 逐字节一致，未做任何修改。
7. 浏览器关闭时若有未保存会话，会弹出"确认离开"提示；点击"取消"可返回页面保存历史。